

Vasic-Digital-Reusable-Module-Suite

Бір рет жаса, барлық жерде пайдалан — Go және KMP модульдерінің жеке сынақтан өткізілген, бір-бірінен тәуелсіз шағын флоты.

Қысқаша мазмұны

`digital.vasic.*` (Go) және Kotlin Multiplatform атты атаулар аясында жарияланған жалпы қолданыстағы, қайта пайдалануға болатын модульдердің үлкен отбасысы. Әрбір модуль дербес, жеке сынақтан өткізіліп, нұсқаға бөлінген және үлкен өнімдердің (Catalogizer, HelixAgent және кеңейтілген флот) тең кодтық базасы ретінде қолданылады. Бұл бет жеке беттер ретінде шу болып көрінетін көптеген шағын құралдарды біріктіреді.

Қысқаша сипаттама

`digital.vasic.*` модульдерінің мұқият іріктелген жинағы — инфрақұрылымдық негіздер (авторизация, кэш, дерекқор, конфигурация, бақылау), AI/агент құрастыру блоктары (RAG, VectorDB, Эмбедингтер, MCP, Агенттік, Жоспарлау) және қорғаныстық-LLM қорғау шаралары (RedTeam, Normalize) — сонымен қатар Kotlin Multiplatform көшірмелерінің жинағы. Әрқайсысы жалпы қолданыстағы, сынақтан өткізілген және қайта пайдалануға болатын болып келеді.

Кеңейтілген сипаттама

`vasic-digital` ұйымы бір құрылымдық қағидаға сүйенеді: «негізгі құжат + көптеген бір-бірінен тәуелсіз, қайта пайдалануға болатын кіші модульдер» философиясы, мұнда жалпы функциялар ешқашан екі рет жазылмайды. Монолиттердің орнына әрбір қайта пайдалануға болатын функция өз алдына шағын модульге — өз репозиторийіне, өз сынақтарына, өз құжаттамасына бөлініп алынады және әрбір тұтынушының

ерекшеліктері ешқашан ішке кіріп кетпеу үшін қатаң түрде бір-бірінен бөлек ұсталады. Бұл бет оларды топтастырады, өйткені жеке алғанда әрқайсысы кітапхана деңгейіндегі құрал болып, жеке өнім беті ретінде шу болып көрінер еді. Біріктіріп алғанда олар ұйымның нағыз күшейткіші болып табылады: «жаңа өнім жасау» дегенді «дәлелденген бөліктерді жинақтауға» айналдыратын жеке инженерлік актив және осы флот дөңгелектерді қайта ойлап таппайды — ол өте жақсы бір дөңгелек жасап, оны барлық жерде пайдаланады деген тұжырымның нақты негізі.

Жинақ үш кластерді қамтиды. **Инфрақұрылымдық негіздер** (Go) әрбір қызметке қажетті құбырларды қамтамасыз етеді: auth (JWT/bcrypt), cache (Redis/TTL), database (миграциялар, екі SQLite/PostgreSQL), config, middleware, observability (Prometheus/OpenTelemetry), ratelimiter, security, storage (S3/MinIO), streaming (WebSocket хабы), eventbus, filesystem (көппротоколдық), discovery/mdns, http3, recovery, concurrency, lazy және тағы басқалары. **AI/агент құрастыру блоктары** (Go) AI жүйелеріне негіз болады: rag, vectordb, embeddings, memory, conversation (шексіз контекстті сығу, іс-шараларды бастау көзі), mcp (Model Context Protocol), toolschema, skillregistry, agentic (графтық жұмыс процестерін басқару), planning (HiPlan/MCTS/Ой ағашы), benchmark (SWE-bench/HumanEval/MMLU), llmops, selfimprove (сыйлықтық модельдеу/RLHF) және toon (Токенге бағытталған объектілер нотациясы). **Қорғаныстық-LLM қорғау шаралары** қарсыластық беріктігін қамтамасыз ететін құралдарды ұсынады: RedTeam (YAML-ға негізделген қарсыластық үлгілер), Normalize (қарсыластық кіріс мәліметтерін стандарттау). **Kotlin Multiplatform** параллельді жинағы негізгі модульдердің көшірмелерін (Auth-KMP, Database-KMP, Security-KMP, UI-Components-KMP және т.б.) көпплатформалық қолданбалар үшін қамтиды.

Мазмұны

Неге құрастырдық

Әр жолы Catalogizer, HelixAgent, Herald және басқа да өнімдерді нөлден бастап шығару – уақытты босқа жою және біркелкілікке жеткізу қиын. Әрбір жалпы мәселені бөлінген, сынақтан өткізілген модульге айналдыру арқылы түзетулер мен жақсартулар бүкіл жүйе бойынша таралады, ал әрбір жаңа өнім сынақтан өткен бөлшектерден құрастырылады.

Неге бұл өзгерістің шегі

Іс жүзінде бұл AI-ға бағытталған бэкэндтерді құруға арналған жеке “стандартты кітапхана” – көптеген командалар осы деңгейді құра алмайды, өйткені олар аутентификацияны, кэшті, RAG инфрақұрылымын бесінші рет қайта шешумен айналысуда. Мұнда инфрақұрылымдық примитивтер, AI құрылыс блоктары және қорғаныстық LLM шектеулер барлығы дайын, жеке сынақтан өткізілген модульдер ретінде қол жетімді. Бұл кішігірім командаға әдетте одан әлдеқайда үлкен топ қажет болатын жылдамдықпен өнімдік жүйелерді шығаруға мүмкіндік береді, ал әдеттегідей қайталаушылық қарызының жинақталуын болдырмайды.

Инновациялық жақтары

- Жүйелік бөліну тәртібі (CONST-051): ішкі модульдер тең кодтық базалар ретінде қарастырылып, тұтынушыға тән ерекшеліктерді алып жүрмейді.
- AI-примитивтер қабаты (RAG, VectorDB, Эмбеддингтер, MCP, ToolSchema, Агенттік, Жоспарлау, LLMOps) қайта пайдалануға арналған модульдер ретінде.
- Қарсыластық төзімділік үшін қорғаныстық LLM шектеу кластері (RedTeam, Normalize).
- Бірдей келісімдерді пайдаланатын параллель Go + Kotlin Multiplatform модульдер жиынтығы.

Мәселелер мен шешімдер

- **Ондаған модульдер арасындағы байланыстың бұзылуын болдырмау:** конституцияның бөліну келісімі және тұтынушыға тән ерекшеліктерді орындау кезінде енгізу арқылы шешілді.
- **Көптеген модульдерді біркелкі және сынақтан өткізу:** модульдер бойынша ортақ келісімдер (әр модульге арналған сынақтар/құжаттамалар/Мәселелер) және HelixConstitution басқару негізі арқылы шешілді.
- **Платформааралық қолжетімділік:** негізгі модульдердің Kotlin Multiplatform көшірмесі арқылы шешілді.

Технологиялық стек (неге және қалай)

- **Go** — модульдердің басым бөлігі (digital.vasic.*).

- **Kotlin Multiplatform** — платформааралық көшірме модульдер (Auth/Database/Security/UI/Concurrency/RateLimiter-KMP).
- **Redis / PostgreSQL / SQLite** — кэш, дерекқор, сақтау примитивтері.
- **Prometheus / OpenTelemetry** — бақылау модулі.
- **WebSocket / HTTP/3 (quic-go) / mDNS** — желілік модульдер.
- **Vector DB / embeddings / RAG / MCP** — AI-примитивтік модульдер.
- **YAML** — RedTeam қарсыластық шаблондары мен конфигурациясы.

ТЕКСЕРІЛМЕГЕН / ЖҰМЫС АЯҚТАЛМАҒАН: бірнеше ұйымдық репозиторийлер өздерін “SCAFFOLD / ЖҰМЫС АЯҚТАЛМАҒАН” деп белгілеген (PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroborous, Claritas, LeakHub, GandalfSolutions). Оларды дайын емес/құрылымдық кезеңдегі деп ұсыну керек, шығарылмаған.